

同济大学计算机系

数据结构课程设计总结报告



学 号 _____

姓 名 _____

专 业 _____ 计算机科学与技术 _____

授课老师 _____

目录

1 课程设计题目与项目概述.....	4
1.1 课程设计题目.....	4
1.2 项目定位.....	4
2 项目背景与相关研究.....	4
2.1 NGS 短读段比对背景.....	4
2.2 K-mer 与 seed-and-extend 思想.....	5
2.3 代表性短读段比对方法.....	5
3 软件功能与需求分析.....	5
3.1 软件功能.....	5
3.2 输入与输出.....	6
3.3 状态与异常需求.....	6
4 总体设计思想.....	6
4.1 分层组织.....	6
4.2 主运行流程.....	7
4.3 软件功能结构图：待补充.....	7
5 系统逻辑结构与物理结构.....	7
5.1 逻辑结构.....	7
5.2 物理结构.....	7
5.3 对象关系与所有权.....	7
6 核心数据结构设计.....	8
6.1 Genome 连续字符数组.....	8
6.2 位置单链表.....	8
6.3 拉链法哈希表.....	9
6.4 结果动态数组.....	9
6.5 核心手写结构与 Qt 容器的边界.....	9
7 核心算法设计与复杂度分析.....	9
7.1 K-mer 索引构建.....	9
7.2 种子候选与汉明距离验证.....	9
7.3 随机 Reference 与 Reads 生成.....	10
7.4 稳定轨道分配.....	10
7.5 时间与空间复杂度.....	10
7.6 必要算法流程图：待补充.....	11
8 系统界面与交互设计.....	11
8.1 整体布局.....	11
8.2 全局概览、局部沙盘与精确详情.....	11
8.3 选择同步与连接关系.....	11
8.4 Reads 与结果面板.....	12
8.5 日志、进度与即时反馈.....	12
9 系统实现与开发平台.....	12
9.1 工程文件组织.....	12
9.2 构建与发布.....	13
10 系统运行结果及分析.....	13

10.1 实际运行与数据来源.....	13
10.2 默认参数检索结果分析.....	14
10.3 K-mer 索引耗时与哈希负载对比实验	14
10.4 边界情况与异常处理.....	15
11 系统安装、运行与操作说明.....	16
11.1 直接运行.....	16
11.2 推荐操作步骤.....	16
11.3 输入文件格式.....	16
11.4 从源码构建.....	16
12 个人思考与进一步改进.....	17
12.1 个人想法与更深层次思考：待补充.....	17
12.2 客观限制与可验证改进方向.....	17
13 总结.....	17
参考文献.....	18

1 课程设计题目与项目概述

1.1 课程设计题目

题目 X：生物信息学大规模 DNA 序列片段高通量错配容错检索引擎。本项目实现的软件名称为“DNA 序列片段错配容错检索引擎”，采用 C++ 与 Qt Widgets 构建可直接运行的桌面图形应用程序。

1.2 项目定位

本项目面向数据结构课程设计，目标是在较小、可观察的数据规模上完整展示“参考基因组预处理—K-mer 索引—种子候选—汉明距离验证—结果可视化”的工作流程。它用于验证手写数据结构和算法设计，不将自身描述为可替代真实生物信息学软件的工业级比对系统。

题目要求参考基因组由 A、T、C、G 构成，长度约为 2000~5000 bp；Read 通过前 K 位种子定位候选位置，再以允许错配数 k 为阈值逐位计算汉明距离。项目当前实现将参数边界集中定义在 core/DnaLimits.h，并由 UI 层 ApplicationLimits.h 直接引用，从而避免界面限制与核心限制分离。

表 1 当前实现的参数范围与默认值

参数	允许范围	默认值	实现说明
Reference 长度	2000~5000 bp	3000 bp	仅允许 A/T/C/G
Read 长度	10~150 bp	30 bp	满足 $Read \leq Reference$
K-mer 长度 K	2~10	6	满足 $K \leq Read$
允许错配数 k	0~2	2	仅替换错配，不处理插入和删除
Reads 数量	1~50 条	50 条	支持单条和批量检索

2 项目背景与相关研究

2.1 NGS 短读段比对背景

下一代测序技术会产生大量短 DNA Reads，将这些 Reads 定位到参考基因组是后续变异分析等工作的基础。Li 与 Homer 的综述将短读段比对方法按哈希表、后缀结构和排序等

索引思路进行归纳，并指出速度、内存占用、敏感性和容错能力之间存在实际权衡^[1]。本课程设计缩小数据规模，但保留了“先建立索引，再批量查询”的核心问题结构。

2.2 K-mer 与 seed-and-extend 思想

BLAT 通过预先建立基因组短词索引来快速发现可能同源的区域，然后进入更细致的比对阶段^[2]。该思想与本项目的“种子候选加完整 Read 验证”直接相关。需要强调的是，BLAT 论文描述的是基因组中非重叠 K-mer 的索引，而本题明确要求参考基因组滑动窗口步长为 1，因此本项目索引的是全部重叠 K-mer，不能把两者的实现细节等同。

Marçais 与 Kingsford 提出的 Jellyfish 说明 K-mer 可以作为哈希表的键，并可围绕大规模 K-mer 处理优化哈希结构^[3]。Jellyfish 实际采用锁无关哈希表和二次探测开放寻址；本项目则依据课程要求手工实现拉链法，并为每个 K-mer 保存位置单链表。因此该文献只用于支撑“K-mer 与哈希表结合”的合理性，而不用于声称两者物理结构相同。

2.3 代表性短读段比对方法

Bowtie 使用 Burrows-Wheeler/FM 索引和质量感知回溯算法在大型基因组上完成允许错配的短序列比对^[4]。与 Bowtie 相比，本项目没有压缩全文索引、质量分数、反向互补链搜索或插入删除处理，而是选择更适合课程展示的 K-mer 拉链哈希表和汉明距离。这种取舍使桶数组、冲突链、位置链表和提前剪枝过程可以被清楚解释和测试。

3 软件功能与需求分析

3.1 软件功能

表 2 功能模块与实际入口

功能模块	界面入口或组件	实际行为
数据准备	导入参考基因组、导入 Reads	读取纯文本或 FASTA，并在成功后更新状态
随机数据	生成参考基因组、随机生成 Reads	两步独立执行；生成 Reads 不自动建索引
索引管理	建立索引	可对同一 Reference 重复构建并刷新指标
检索	检索当前 Read、高通量批量检索	要求已有有效索引，不自动建索引

功能模块	界面入口或组件	实际行为
结果浏览	Reads 列表、检索结果、候选位置表	按稳定 Read ID 联动选择
可视化	全局概览、局部沙盘、精确详情	定位区间、显示错配和选中连接关系
结果输出	导出结果	生成 UTF-8 CSV，保留全部匹配位置
反馈	运行日志、进度条、Snackbar	记录流程并对成功或错误即时提示

3.2 输入与输出

Reference 支持单行或多行纯文本以及 FASTA；Reads 支持每行一条的纯文本或 FASTA。SequenceFileIO 使用 UTF-8 文本流读取，去除 UTF-8 BOM、空格、制表符和回车，并统一转换为大写。Reference 必须形成一条非空 DNA 序列；Reads 文件中的每条记录都必须非空且只能包含 A、T、C、G。

CSV 表头依次为“Read 编号、序列、匹配排名、匹配位置、汉明距离、错配详情、状态、候选位置数、耗时（微秒）”。一个 Read 存在多个合法位置时逐行输出，不只保留最佳位置；没有成功匹配时输出“未命中”记录。文本字段通过双引号包围并将内部双引号转义，检索耗时以微秒为单位保留两位小数。

3.3 状态与异常需求

当前交互流程要求用户显式建立索引。若未建立索引，或 Reference、K 变化使索引失效，检索入口返回明确错误；只改变允许错配数时保留索引但清空旧结果。导入新 Reference 会清除旧 Reads、索引和结果；导入新 Reads 保留有效索引但清除结果。程序关闭时 MainWindow 析构函数清空输入、Reference、会话并释放 SearchEngine，因此重新启动不会恢复上一次运行状态。

4 总体设计思想

4.1 分层组织

系统分为核心算法层和 Qt GUI 层。core 目录只依赖 C++ 标准基础设施，负责 Genome、PositionList、ZipperHashTable、ReadGenerator、SearchResult 和 SearchEngine；ui 目录负责

文件对话框、状态模型、表格、绘制、日志和用户反馈。MainWindow 通过 SearchEngine 的公开接口建立索引和检索，UI 不直接访问哈希桶、HashEntry 或 PositionList。

4.2 主运行流程

典型流程为：生成或导入 Reference；生成或导入 Reads；设置 K 和允许错配数；点击“建立索引”；选择单条 Read 或执行 50 条批量检索；在全局概览、局部沙盘和右侧表格中浏览结果；需要时导出 CSV。每次建索引和检索均在日志中记录开始与完成信息，使算法阶段能够在演示中被观察。

4.3 软件功能结构图：待补充

待补充。此处应由本人绘制软件功能结构图，建议至少展示“数据输入与生成、索引管理、单条/批量检索、结果模型、三层可视化、日志反馈、CSV 导出”之间的关系。

5 系统逻辑结构与物理结构

5.1 逻辑结构

逻辑上，Reference 是一条线性 DNA 序列；每个长度为 K 的滑动窗口形成一个 K-mer 键；相同键对应其全部出现位置的线性链表；不同键通过哈希函数映射到桶，映射到同一桶的不同键再构成冲突链。查询时，Read 的前 K 位形成种子，种子对应的位置链表就是候选集合。

5.2 物理结构

Genome 以 new char[length+1] 申请连续存储，末尾写入 '\0'。哈希表以 new HashEntry*[capacity] 申请连续桶数组，每个桶保存 HashEntry 指针。HashEntry 内含深拷贝的 Genome 键、位置链表头指针和下一冲突节点指针。位置链表节点 ListNode<int> 分散申请，通过 _next 串联；新位置采用头插法。

5.3 对象关系与所有权

SearchEngine 拥有 Reference、ZipperHashTable、当前 K、错配阈值和最近一次 SearchResult。ZipperHashTable 析构时逐桶删除 HashEntry，HashEntry 析构时逐节点释放位置链表，最后释放桶数组。SearchResult 和 SearchMatch 对匹配数组与错配偏移数组执行深拷贝。该所有权设计避免结果对象引用已经释放的临时 Read 缓冲区。

6 核心数据结构设计

表 3 核心类与手写存储

类型	关键成员	职责
Genome	int _length; char* _bases	连续字符数组、终止符、深拷贝和逐字符相等判断
ListNode<int>	int _data; ListNode* _next	保存一个 K-mer 的出现位置
HashEntry	Genome _key; PositionList _positions; HashEntry* _next	保存键、位置链表和桶内冲突链
ZipperHashTable	HashEntry** _buckets; 容量及统计量	动态桶数组、拉链冲突处理、插入/查找/清空
SearchMatch	位置、错配数、int* 偏移数组	保存一个成功匹配及其独立错配位置
SearchResult	Read、动态 Match 数组、候选数和耗时	保存一次检索的完整结果
ReadRecord / ReadBatch	Genome、来源位置、最多两个突变偏移；动态记录数组	保存随机测试数据及深拷贝

6.1 Genome 连续字符数组

Genome 的默认对象也分配 1 字节并写入 '\0'，因此 Bases() 始终返回可用的 C 风格字符串缓冲区。构造函数按给定长度复制字符；复制构造与赋值运算符重新申请缓冲区，赋值时先准备新数组再释放旧数组。它没有使用 std::string，满足题目对核心 Reference 连续存储和深拷贝的要求。

6.2 位置单链表

PositionList 是 ListNode<int>* 的别名。InsertPosition 创建节点，将 _next 指向原链表头，再更新头指针。头插法为 $O(1)$ ，适合索引构建期间持续追加位置。其副作用是同一 K-mer 的位置链表按构建顺序逆序保存，但检索结果进入 SearchSessionModel 后会按汉明距离和起点排序，因此对外最佳位置规则仍然确定。

6.3 拉链法哈希表

ZipperHashTable 默认由 SearchEngine 使用容量 8191 的桶数组。Hash 函数将 A/C/G/T 映射为 1/2/3/4，以 131 为乘数迭代取模。Insert 先在目标桶的冲突链中查找相同键；若不存在，则创建 HashEntry 并头插到桶链；随后将位置头插到该键的位置链表。
_keyCount、_positionCount、_collisionCount 和负载因子用于界面指标展示。

6.4 结果动态数组

SearchResult 未使用 std::vector，而是以 SearchMatch* 管理动态数组。容量从 0 增长到 4，此后按 2 倍扩容；扩容时逐项深拷贝。SearchMatch 按实际错配数保存偏移数组，使汉明距离为 2 的结果只记录两个离散错配位置，避免出现错配后后续字符整体被标错的现象。

6.5 核心手写结构与 Qt 容器的边界

全局搜索显示 core 目录没有 std::string、std::vector、std::list、std::map、std::unordered_map、QHash、QMap、QVector 或 QList。Qt 的 QVector、QList 和 QString 仅出现在 ui 目录，用于读取文件、视图模型、表格数据、轨道布局和绘制线段。这些容器不承担 K-mer 索引、桶冲突链或位置链表功能，因此没有替代课程要求的手写核心数据结构。

7 核心算法设计与复杂度分析

7.1 K-mer 索引构建

IndexConstruct 令最后起点为 N-K，从 position=0 到 position=N-K 逐位滑动。每次以 Reference.Bases()+position 构造长度为 K 的 Genome 键，并调用 ZipperHashTable::Insert。因而处理位置总数严格为 N-K+1，首位置为 0，末位置为 N-K，步长恒为 1。以 3000 bp、K=6 为例，索引位置数应为 2995。

7.2 种子候选与汉明距离验证

Search 首先校验 Read 长度、字符集合以及 $K \leq \text{Read} \leq \text{Reference}$ ，再截取前 K 位作为 seed。Find 返回 seed 的位置链表。对每个候选起点，算法先排除越过 Reference 末端的区间，再从偏移 0 开始比较等长字符。遇到不同字符时记录偏移并增加 mismatchCount；一

且 mismatchCount>k，立即停止当前候选。只有最终错配数不超过阈值的候选才追加到 SearchResult。

本题采用汉明距离，未实现插入和删除，因此不存在字符整体偏移。每个错配偏移都是独立的同列差异；UI 根据这些偏移绘制黄色或红色单元格，并只在匹配位置绘制连接线。

7.3 随机 Reference 与 Reads 生成

GenerateReference 使用 mt19937 和均匀分布从 A/C/G/T 中生成指定长度序列。Generate 为每条 Read 随机选择合法来源起点，复制连续 readLength 个字符，然后在 [K, readLength-1] 区间产生 1~2 个不重复突变偏移，并保证替换碱基不同于原字符。前 K 位种子不被破坏。当 Read 长度恰好等于 K 时没有可突变区域，实现按设计生成 0 处突变。

7.4 稳定轨道分配

GenomeAlignmentWidget 在检索完成后统一收集全部匹配区间，按起点、Read ID、matchIndex 和 readIndex 排序，再将每个区间放入不发生重叠的最低可用轨道。轨道编号存入 m_readLayout；水平滚动和缩放只改变坐标映射，不重新分配轨道。只有重新检索、生成/导入数据、参数导致结果失效或清空结果时才重建布局。

7.5 时间与空间复杂度

表 4 主要算法复杂度

过程	时间复杂度	空间复杂度	说明
索引构建	平均 $O((N-K+1) \cdot K)$	$O(B+D \cdot K+N-K+1)$	B 为桶数，D 为不同 K-mer 数
单条检索	平均 $O(K+C \cdot L)$ ，最坏 $O(K+C \cdot L)$	$O(k+M \cdot k)$	C 为种子候选数，L 为 Read 长度，M 为成功位置数
Reference 生成	$O(N)$	$O(N)$	生成后由 Genome 持有连续数组
R 条 Reads 生成	$O(R \cdot L)$	$O(R \cdot L)$	每条最多 2 个突变
轨道布局	$O(M \log M)$	$O(M)$	排序后使用最低可用轨道

提前剪枝不会改变最坏复杂度,但当错误候选较早超过阈值时可以减少实际比较字符数。
K 较小时种子重复概率增加,候选 C 通常增大;K 较大时候选减少,但若 Read 前 K 位存在真实突变,单前缀种子方案会漏掉正确位置。这也是随机生成器保护前 K 位的原因。

7.6 必要算法流程图：待补充

待补充。此处应由本人绘制至少两张流程图：K-mer 索引构建流程,以及“读取前 K 位种子—遍历候选—逐字符比较—超阈值剪枝—记录结果”的检索流程。

8 系统界面与交互设计

8.1 整体布局

主窗口采用深色 Fusion 风格,初始尺寸 1640×980,最小尺寸 1280×760。顶部保留导入 Reference、导入 Reads 和导出结果;左侧集中参数设置、生成 Reference、生成 Reads、建立索引、批量检索、当前 Read 检索和清空结果;中间为全局概览、局部沙盘和精确详情;右侧为 Reads/检索结果双页签以及当前 Read 的全部匹配位置;底部为可上下调整 72~140 像素的运行日志区域。

8.2 全局概览、局部沙盘与精确详情

全局概览用颜色区分 0、1、2 处错配的成功区间,并显示当前视口。局部沙盘按碱基单元格绘制 Reference 和 Reads,支持拖动、滚动、Ctrl+滚轮缩放、适应全局和重置视图。Read 整体以成功底色和边框表达命中,只有实际错配单元格使用黄色或红色。精确详情只保留 Reference、Read 和“比对关系”三行,错配单元格显示碱基变化及绝对位置,三行共享横向滚动。

8.3 选择同步与连接关系

用户可从局部沙盘、Reads 列表或检索结果表选择 Read, SearchSessionModel 以稳定 Read ID 同步三处状态。默认检索完成后沙盘不自动选中 Read;只有显式选择某一成功位置时才绘制该 Read 与 Reference 的逐碱基细线。错配位置不绘线;连接线按被其他 Read 占据的纵向范围切成多个线段并绕经空隙,且绘制层级低于字符和错配高亮。点击沙盘空白区域会清除选择和全部连接线。

8.4 Reads 与结果面板

Reads 列表展示 Read ID、序列预览、长度和突变数，并在下方显示完整序列、来源位置 and 突变变化。单击一行会填入左侧单条检索框并更新当前候选表；双击只触发一次单条检索。批量完成后自动切换到检索结果页；结果按最佳位置、距离和状态展示，当前 Read 的匹配位置区域始终随选择更新。

8.5 日志、进度与即时反馈

运行日志最多保留 200 个文本块，明确记录索引构建和检索的开始、完成、参数、候选数、匹配数及微秒耗时。Snackbar 从窗口左下方向右进入，可同时堆叠最多 5 条；成功消息约 1.5 秒、错误消息约 2 秒后向上淡出。相同消息在短时间内重复出现时复用现有 Snackbar 并重启计时，以降低动画堆积开销。

9 系统实现与开发平台

表 5 已核验的开发与构建环境

项目	实际环境
操作系统	Windows 11 x64
开发语言	C++17
GUI 框架	Qt 6.11.1 Widgets, Fusion 风格
编译器	MinGW-w64 GCC 13.1.0
构建工具	CMake 3.30.5、Ninja 1.12.1
发布工具	windeployqt, 附带 Qt Release DLL 和 MinGW runtime

9.1 工程文件组织

CMakeLists.txt 将 core 下文件编译为静态库 DnaCore，再由 DNA_SearchEngine 可执行程序链接 DnaCore 和 Qt6::Widgets。core 与 ui 在构建层面分离；main.cpp 仅创建 QApplication、设置应用名称、Fusion 风格和 Microsoft YaHei UI 字体，然后显示 MainWindow。

9.2 构建与发布

项目提供 `configure-debug.cmd`、`build-debug.cmd`、`configure-release.cmd`、`build-release.cmd` 和 `deploy-release.cmd`。`qt-env.cmd` 目前为本机 Qt 安装提供 `D:\Qt` 默认路径，同时允许通过 `QT_PREFIX`、`DNA_MINGW_BIN` 等环境变量覆盖。`deploy-release.cmd` 使用临时 `staging` 目录运行 `windeployqt`，检查关键 `DLL` 和 `platforms/qwindows.dll` 后再原子替换现有发布目录，失败时保留原发布版本。

10 系统运行结果及分析

10.1 实际运行与数据来源

本次实验直接运行提交目录中的 `Release` 版 `DNA_SearchEngine.exe`，导入配套的 `test_reference.fasta` 和 `test_reads.txt`。`Reference` 长度为 `3000 bp`，共导入 `50` 条长度为 `30 bp` 的 `Reads`；允许错配数固定为 `k=2`，只改变 `K-mer` 长度 `K`。对 `K=2`、`6`、`10` 分别显式建立索引并执行一次高通量批量检索，随后导出 `DNA_Search_Results_K2.csv`、`DNA_Search_Results_K6.csv` 和 `DNA_Search_Results_K10.csv`。不同 `K-mer` 数、建索引耗时和哈希负载取自同次运行界面，候选数、成功数、汉明距离及检索耗时由相应 `CSV` 逐行统计。

表 6 实际运行数据与记录依据

记录项	实际值	记录依据
测试程序	提交执行目录中的 <code>DNA_SearchEngine.exe</code>	<code>Release</code> 发布程序实际运行
Reference	<code>test_reference.fasta</code> ， <code>3000 bp</code>	导入完成界面与运行统计
Reads	<code>test_reads.txt</code> ， <code>50</code> 条，每条 <code>30 bp</code>	<code>Reads</code> 列表与三份 <code>CSV</code>
固定参数	允许错配数 <code>k=2</code> ， <code>Reads</code> 数量 <code>50</code>	三次运行均保持不变
变量	<code>K=2</code> 、 <code>K=6</code> 、 <code>K=10</code>	每次修改 <code>K</code> 后重新建立索引
结果文件	<code>DNA_Search_Results_K2/K6/K10.csv</code>	<code>UTF-8 CSV</code> ，每个文件 <code>50</code> 个 <code>Read</code> 记录

记录项	实际值	记录依据
开发与运行环境	Windows 11 x64, Qt 6.11.1 MinGW 64-bit	项目已核验开发环境与本机运行环境

10.2 默认参数检索结果分析

默认参数 $K=6$ 、 $k=2$ 时, 50 条 Reads 全部检索成功, 成功数为 50, 失败数为 0。CSV 共 50 条结果记录, 每条 Read 恰好对应 1 个合法匹配位置, 没有重复或多位置结果。汉明距离分布为 $d=1$ 的 25 条、 $d=2$ 的 25 条, 与测试数据中交替设置 1~2 个替换突变的生成规则一致。每条结果的错配详情均给出了参考基因组绝对位置和碱基替换关系, 可与界面的精确比对详情逐项对应。

$K=6$ 时共验证 84 个种子候选, 平均每条 Read 为 1.68 个, 单条候选数范围为 1~5。50 条 Reads 的累计检索耗时为 35.00 μs , 平均每条 0.70 μs , 单条耗时范围为 0.40~1.50 μs 。 $K=6$ 的前缀种子在生成测试数据时受到保护, 因此正确位置均能进入候选集合; 后续完整 Read 验证准确保留了 1 处或 2 处错配的结果, 说明“种子筛选—汉明距离验证—超阈值剪枝”的流程在默认参数下与预期一致。

10.3 K-mer 索引耗时与哈希负载对比实验

对比实验保持 Reference、Reads、Read 长度和允许错配数不变, 依次将 K 设置为 2、6、10。每次修改 K 后旧索引自动失效, 再重新点击“建立索引”, 最后执行同一批 50 条 Reads 的批量检索。长度为 $N=3000$ 的 Reference 在三种 K 下分别处理 $N-K+1$, 即 2999、2995、2991 个滑动窗口; 索引和检索统计见表 7。

表 7 不同 K 值的索引与检索结果对比

K	索引位置数	不同 $K\text{-mer}$	建索引耗时/ μs	哈希负载	候选总数 / 检索耗时 / 成功数
2	2999	16	156	0.002	9448 / 465.70 μs / 50
6	2995	2128	463	0.260	84 / 35.00 μs / 50
10	2991	2987	650	0.365	37 / 33.60 μs / 37

随着 K 从 2 增大到 10, 不同 $K\text{-mer}$ 数由 16 增至 2987, 哈希负载由 0.002 增至 0.365, 建索引耗时由 156 μs 增至 650 μs 。原因是 $K=2$ 只有 $4^2=16$ 种可能短词, 大量滑动

窗口被归并到少数键的位置链表；K 增大后短词区分度提高，独立键数量显著增加，需要创建和插入更多哈希节点，因此本次实测的建索引耗时随之上升。

候选规模的变化更为明显：K=2 平均每条 Read 产生 188.96 个候选，累计 9448 个候选，批量检索耗时 465.70 μs；K=6 仅产生 84 个候选，累计耗时降至 35.00 μs。两种参数均成功检索 50 条 Reads，且对应匹配位置完全一致。这说明在本组数据上，K=6 在保持召回率的同时显著减少了完整 Read 的逐字符验证次数，是三组参数中较合适的取值。

K=10 时仅产生 37 个候选，检索耗时为 33.60 μs，但成功数下降为 37。逐行核对发现，未命中的 Read 编号为 1、8、15、16、18、22、25、32、39、40、42、46、49；这 13 条 Read 在相对偏移 6~9 处均存在突变，恰好进入 K=10 的前缀种子范围。该批测试数据按默认 K=6 生成，只保护前 6 位种子，因此不能保证前 10 位不发生突变。K=10 已命中的 37 条 Read 与 K=6 的匹配位置完全一致。由此可见，较大的 K 能继续压缩候选集合，但单一前缀种子对种子区突变敏感；若要公平评价 K=10 的召回率，应使用保护前 10 位的 Reads，或在后续改进中采用多种子策略。

10.4 边界情况与异常处理

系统围绕课程题目的主要边界进行了针对性核对。下表只保留与算法正确性和用户操作直接相关的场景，不展开工程化压力测试或外部内存检测工具结果。

表 8 典型边界与异常场景

测试场景	预期结果	实际表现
完全匹配、1 处和 2 处错配	k=0 仅接受完全匹配；k=1 接受至多 1 处；k=2 接受至多 2 处	符合阈值定义，错配位置独立记录
Reference 首尾合法起点	起点 0 与 N-ReadLength 均可检索，末字符参与比较	均可返回，不越过 Reference 边界
重复 K-mer 与多候选位置	同一键保存全部出现位置，逐个验证并保留全部合法结果	候选不覆盖，结果按距离和起点稳定排序
空文件、非法碱基或格式错误	拒绝导入并给出明确原因，不保留半加载数据	通过 Snackbar、状态栏或文件错误信息提示
缺少 Reference、Reads 或有效索引	阻止对应操作并提示缺失条件	检索入口不会在状态不完整时继续执行

测试场景	预期结果	实际表现
修改 K	旧索引自动失效；再次建立索引时按新 K 重建	旧结果清除，未重建前检索会提示索引无效
重新导入或生成数据	清除旧检索结果和当前选中状态	Reference 更新时同时清索引；Reads 更新时保留有效索引

11 系统安装、运行与操作说明

11.1 直接运行

进入“250021_张三_计算机技术_执行程序”目录，双击 DNA_SearchEngine.exe。必须保留 EXE、Qt DLL、MinGW DLL、platforms、imageformats、styles 等插件目录的相对位置，不能只复制 EXE。程序不依赖 Qt Creator，正常发布包无需重新编译。

11.2 推荐操作步骤

第一步，点击“生成参考基因组”或导入合法 Reference。第二步，点击“随机生成 Reads”或导入 Reads。第三步，设置 K 和允许错配数并点击“建立索引”。第四步，在 Reads 列表中选择一条 Read 执行单条检索，或点击“高通量批量检索”。第五步，在全局概览、局部沙盘、检索结果和候选位置表中选择结果。第六步，按需点击“导出结果”保存 CSV。

若改变 K，必须重新点击“建立索引”；改变允许错配数只需重新检索。相同 Reference 允许重复建立索引，每次都会刷新构建时间和哈希指标。清空检索结果不会删除 Reference、Reads 或有效索引。

11.3 输入文件格式

Reference 文件可以是 FASTA，也可以是只含 A/T/C/G 的纯文本，多行会拼接为一条序列。Reads 文件可以是 FASTA，或每行一条 Read 的纯文本。随发布材料提供 test_reference.fasta 和 test_reads.txt，用于验证导入、建索引、单条检索和批量检索；使用它们时推荐保持 K=6、k=2、Read 长度 30。

11.4 从源码构建

在已安装 Qt 6.11.1 MinGW 64-bit、匹配的 GCC、CMake 和 Ninja 的环境中，可先运行 tools/configure-release.cmd，再运行 tools/build-release.cmd。构建成功后运行 tools/deploy-

release.cmd, 由 windeployqt 复制依赖。若 Qt 未安装在 D:\Qt, 应先设置 QT_PREFIX、DNA_MINGW_BIN、DNA_CMAKE 和 DNA_NINJA, 或修改 qt-env.cmd 的默认路径。

12 个人思考与进一步改进

12.1 个人想法与更深层次思考：待补充

待补充。此处保留给本人结合课程学习、数据结构取舍、调试过程和展示体验撰写个人思考, 本文档不代写。

12.2 客观限制与可验证改进方向

当前只使用 Read 前 K 位作为唯一种子, 因此正确位置的前 K 位发生突变时可能没有候选。可以在不改变核心课程要求的前提下研究多种子或分段种子, 但需重新分析候选去重与时间空间代价。当前随机生成器通过保护前 K 位确保测试 Read 可被该算法召回, 这是一项明确的实验前提。

当前比对只计算等长字符的汉明距离, 不能处理插入、删除、反向互补链和碱基质量分。Bowtie 等成熟工具采用更复杂的压缩索引和回溯策略, 本项目不应夸大真实 NGS 工作负载。若扩展功能, 应先保持现有手写结构的可验证性, 再分别引入反向互补查询、多个种子和可选的带状动态规划。^[4]

GUI 当前在主线程中执行批量检索, 并通过进度条和事件处理维持反馈。课程规定的规模能够正常完成; 若扩大 Reference 或 Reads 数量, 应将不可变索引与后台任务结合, 同时设计取消、进度和结果提交边界, 避免 UI 模型与核心对象跨线程悬空。

13 总结

本课程设计围绕题目 X 完成了可运行的 C++/Qt 桌面程序。核心层手工实现连续 Genome 字符数组、动态桶数组、HashEntry 拉链节点和 K-mer 位置单链表; 索引按步长 1 覆盖 N-K+1 个位置; 检索以 Read 前 K 位产生候选, 再通过汉明距离和超阈值提前剪枝验证。

系统将数据生成、文件导入、显式索引构建、单条/批量检索、CSV 导出、全局概览、局部序列沙盘、精确对齐和 SnackBar 日志反馈组织为完整流程。边界与异常处理围绕错配阈值、首尾位置、重复 K-mer、非法输入和状态失效进行说明; 系统仍受单前缀种子、仅支持替换错配和课程规模限制, 这些边界已在报告中明确。

参考文献

- [1] LI H, HOMER N. A survey of sequence alignment algorithms for next-generation sequencing[J]. Briefings in Bioinformatics, 2010, 11(5): 473-483. DOI: 10.1093/bib/bbq015.
- [2] KENT W J. BLAT-The BLAST-like alignment tool[J]. Genome Research, 2002, 12(4): 656-664. DOI: 10.1101/gr.229202.
- [3] MARÇAIS G, KINGSFORD C. A fast, lock-free approach for efficient parallel counting of occurrences of k-mers[J]. Bioinformatics, 2011, 27(6): 764-770. DOI: 10.1093/bioinformatics/btr011.
- [4] LANGMEAD B, TRAPNELL C, POP M, et al. Ultrafast and memory-efficient alignment of short DNA sequences to the human genome[J]. Genome Biology, 2009, 10(3): R25. DOI: 10.1186/gb-2009-10-3-r25.